CTO Portal → Technology Executive Dashboard Migration Guide

Version: 1.0
Status: In Progress
Target Completion: Q1 2025
Last Updated: 2025-01-XX

Overview

This guide provides step-by-step instructions for migrating the existing CTO Portal (`/cto`) to use the TPI UI/UX design system, transforming it into the "Technology Executive Dashboard" with consistent, enterprise-grade UI/UX.

Current State Analysis

Existing Structure

Component: `src/pages/CTOPortal.tsx`
Layout: `ExecutivePortalLayout` (Mantine-based)
Navigation: Tab-based navigation with 20+ sections

Current Sections:

- CTO Evaluation Gate
- CTO Onboarding & Governance
- Training
- CTO Command Center (Dashboard)
- Advanced Infrastructure
- DevOps & CI/CD
- Security & Compliance
- Team & Resources
- Technology Roadmap
- Tech Cost Management
- Morning Review
- Sprint Management
- Code Reviews
- IT Help Desk
- Code Editor
- Developer Onboarding
- Incidents
- Assets
- Executive Communications
- Draft Documents

- Articles Generator
- Instruction Manual

Current Issues

1. **Inconsistent Layout:** Uses `ExecutivePortalLayout` (different from TPI standard)
2. **Mixed Patterns:** Some sections use different UI patterns
3. **No Standard Components:** Custom components not following TPI standards
4. **Navigation:** Tab-based, not sidebar-based
5. **Spacing:** Inconsistent spacing and padding
6. **Tables:** Not using TPI `DataTable` component
7. **Drawers:** Not using TPI `DetailDrawer` pattern

Migration Strategy

Phase 1: Foundation (Week 1-2)

Goal: Replace layout and navigation

Tasks:

1. Create `PortalLayout` component (if not exists)
2. Replace `ExecutivePortalLayout` with `PortalLayout`
3. Convert tab navigation to sidebar navigation
4. Implement `TopBar` component
5. Add `Breadcrumbs` to sub-pages

Files to Modify:

- `src/pages/CTOPortal.tsx`
- Create: `src/components/tpi/PortalLayout.tsx`
- Create: `src/components/tpi/TopBar.tsx`
- Create: `src/components/tpi/SideNav.tsx`

Phase 2: Components (Week 3-4)

Goal: Replace custom components with TPI components

Tasks:

1. Replace custom tables with `DataTable`
2. Replace modals with `DetailDrawer` where appropriate
3. Add `KpiCard` components to dashboard
4. Implement `StatusBadge` for status indicators
5. Add `EmptyState` and `ErrorState` components
6. Implement `SkeletonLoader` for loading states

Files to Modify:

- All CTO portal component files

- Create: `src/components/tpi/DataTable.tsx`
- Create: `src/components/tpi/DetailDrawer.tsx`
- Create: `src/components/tpi/KpiCard.tsx`
- Create: `src/components/tpi/StatusBadge.tsx`
- Create: `src/components/tpi/EmptyState.tsx`
- Create: `src/components/tpi/ErrorState.tsx`
- Create: `src/components/tpi/SkeletonLoader.tsx`

Phase 3: Pages (Week 5-6)

Goal: Apply TPI page templates

Tasks:

1. Apply Dashboard Template to CTO Command Center
2. Apply Work Management Template to Incidents, Assets
3. Apply Kanban Template to Sprint Management
4. Apply Detail Template to individual entity pages
5. Apply Admin Settings Template to portal settings

Files to Modify:

- `src/components/cto/EnhancedCTODashboard.tsx`
- `src/components/cto/IncidentsDashboard.tsx`
- `src/components/cto/AssetManagement.tsx`
- `src/components/cto/SprintManagement.tsx`

Phase 4: Content & Polish (Week 7-8)

Goal: Apply content standards and accessibility

Tasks:

1. Update all copy to TPI content style guide
2. Ensure WCAG AA compliance
3. Add ARIA labels and semantic HTML
4. Test keyboard navigation
5. Test screen readers
6. Performance optimization

Files to Review:

- All CTO portal files
- Run accessibility audits
- Performance testing

Step-by-Step Migration

Step 1: Create TPI Component Library

Create Base Components

1. **PortalLayout**

```
// src/components/tpi/PortalLayout.tsx
import { SideNav } from './SideNav';
import { TopBar } from './TopBar';
import { Breadcrumbs } from './Breadcrumbs';

interface PortalLayoutProps {
  portalName: string;
  sidebarItems: SidebarItem[];
  user: User;
  children: React.ReactNode;
}

export function PortalLayout({ portalName, sidebarItems, user, children }:
PortalLayoutProps) {
  // Implementation following TPI spec
}
```

2. **SideNav**

```
// src/components/tpi/SideNav.tsx
export function SideNav({ items, activePath, onNavigate }: SideNavProps) {
  // Implementation following TPI spec
}
```

3. **TopBar**

```
// src/components/tpi/TopBar.tsx
export function TopBar({ portalName, onSearch, user }: TopBarProps) {
  // Implementation following TPI spec
}
```

Step 2: Update CTO Portal Root

Replace Layout

```
// src/pages/CTOPortal.tsx
// BEFORE:
import ExecutivePortalLayout from '@/components/executive/ExecutivePortalLayout';

<ExecutivePortalLayout>
  {/* content */}
</ExecutivePortalLayout>
```

```
// AFTER:
import { PortalLayout } from '@/components/tpi/PortalLayout';
import { SideNav } from '@/components/tpi/SideNav';
import { TopBar } from '@/components/tpi/TopBar';

<PortalLayout
  portalName="Technology Executive Dashboard"
  sidebarItems={ctoSidebarItems}
  user={currentUser}
>
  {/* content */}
</PortalLayout>
```

Convert Navigation

```
// Convert tab-based navigation to sidebar items
const ctoSidebarItems = [
  {
    id: 'dashboard',
    label: 'CTO Command Center',
    icon: BarChart3,
    path: '/cto',
    children: [
      { id: 'overview', label: 'Overview', path: '/cto' },
      { id: 'morning-review', label: 'Morning Review', path: '/cto/morning-review'
}
    ]
  },
  {
    id: 'infrastructure',
    label: 'Infrastructure',
    icon: Cloud,
    path: '/cto/infrastructure',
    children: [
      { id: 'advanced', label: 'Advanced Infrastructure', path:
'/cto/infrastructure/advanced' },
      { id: 'devops', label: 'DevOps & CI/CD', path: '/cto/infrastructure/devops'
}
    ]
  },
  {
    id: 'incidents',
    label: 'Incidents',
    icon: Bug,
    path: '/cto/incidents'
  },
  // ... more items
];
```

Step 3: Update Dashboard

Apply Dashboard Template

```
// src/components/cto/EnhancedCTODashboard.tsx
// BEFORE: Custom layout
// AFTER: TPI Dashboard Template

import { PageHeader } from '@/components/tpi/PageHeader';
import { KpiCard } from '@/components/tpi/KpiCard';
import { ActivityFeed } from '@/components/tpi/ActivityFeed';

export function EnhancedCTODashboard() {
  return (
    <>
      <PageHeader
        title="CTO Command Center"
        description="Overview of platform health and operations"
        actions={
          <>
            <Button variant="outline">Export Report</Button>
            <Button>New Incident</Button>
          </>
        }
      />

      <div className="grid grid-cols-4 gap-4 mb-6">
        <KpiCard
          label="System Uptime"
          value={99.97}
          format="percentage"
          trend={{ value: 0.02, direction: "up", period: "vs last month" }}
        />
        {/* More KPI cards */}
      </div>

      <ActivityFeed activities={recentActivities} realTime />
    </>
  );
}
```

Step 4: Update Incidents Page

Apply Work Management Template

```
// src/components/cto/IncidentsDashboard.tsx
// BEFORE: Custom table implementation
// AFTER: TPI Work Management Template

import { PageHeader } from '@/components/tpi/PageHeader';
```

```
import { Breadcrumbs } from '@/components/tpi/Breadcrumbs';
import { FilterBar } from '@/components/tpi/FilterBar';
import { DataTable } from '@/components/tpi/DataTable';
import { DetailDrawer } from '@/components/tpi/DetailDrawer';

export function IncidentsDashboard() {
  return (
    <>
      <PageHeader
        title="Incident Management"
        description="Track and resolve platform incidents"
        actions={<Button>New Incident</Button>}
      />
      <Breadcrumbs items={[
        { label: "Dashboard", path: "/cto" },
        { label: "Incidents" }
      ]} />

      <FilterBar
        filters={incidentFilters}
        activeFilters={activeFilters}
        onFilterChange={setFilters}
      />

      <DataTable
        data={incidents}
        columns={incidentColumns}
        onRowClick={(row) => setSelectedIncident(row.id)}
        exportable
      />

      <DetailDrawer
        open={!!selectedIncident}
        onClose={() => setSelectedIncident(null)}
        title="Incident Details"
        entityId={selectedIncident}
      >
        {/* Drawer content */}
      </DetailDrawer>
    </>
  );
}
```

Step 5: Update Content

Apply Content Style Guide

```
// BEFORE:
<EmptyState
  title="No incidents"
  description="You don't have any incidents yet."
```

```
/>

// AFTER:
<EmptyState
  title="No incidents found"
  description="Get started by creating your first incident report."
  action={{
    label: "New Incident",
    onClick: handleCreate
  }}
/>
```

Step 6: Add Accessibility

Add ARIA Labels

```
// Add to all interactive elements
<Button
  aria-label="Create new incident"
  onClick={handleCreate}
>
  New Incident
</Button>

<DataTable
  aria-label="Incidents table"
  data={incidents}
  columns={columns}
/>
```

Ensure Keyboard Navigation

- Test all interactions with keyboard only
- Add focus indicators
- Implement focus trapping in modals/drawers

Migration Checklist

Layout & Navigation

- [ ] Replace `ExecutivePortalLayout` with `PortalLayout`
- [ ] Convert tab navigation to sidebar
- [ ] Implement `TopBar` component
- [ ] Add `Breadcrumbs` to sub-pages
- [ ] Update routing structure

Components

- [ ] Replace custom tables with `DataTable`
- [ ] Replace modals with `DetailDrawer` (where appropriate)
- [ ] Add `KpiCard` to dashboard
- [ ] Implement `StatusBadge` for statuses
- [ ] Add `EmptyState` components
- [ ] Add `ErrorState` components
- [ ] Implement `SkeletonLoader` for loading

Pages

- [ ] Apply Dashboard Template to Command Center
- [ ] Apply Work Management Template to Incidents
- [ ] Apply Work Management Template to Assets
- [ ] Apply Kanban Template to Sprint Management
- [ ] Apply Detail Template to entity pages
- [ ] Apply Admin Settings Template to settings

Content & Accessibility

- [ ] Update all copy to TPI style guide
- [ ] Add ARIA labels to all interactive elements
- [ ] Ensure keyboard navigation works
- [ ] Test with screen readers
- [ ] Verify color contrast (WCAG AA)
- [ ] Add focus indicators
- [ ] Test responsive design

Testing

- [ ] Unit tests for new components
- [ ] Integration tests for page flows
- [ ] Accessibility audit (axe, WAVE)
- [ ] Performance testing
- [ ] User acceptance testing

Rollout Plan

Phase 1: Internal Testing (Week 1-2)

- Deploy to staging environment
- Internal team testing
- Fix critical issues

Phase 2: Beta Testing (Week 3-4)

- Limited user group
- Collect feedback
- Iterate on issues

Phase 3: Gradual Rollout (Week 5-6)

- 25% of users
- Monitor metrics
- Fix issues

Phase 4: Full Rollout (Week 7-8)

- 100% of users
- Monitor closely
- Document lessons learned

Rollback Plan

If Critical Issues Found:

1. Revert to previous version immediately
2. Document issues
3. Fix in staging
4. Re-test before re-deployment

Rollback Triggers:

- Data loss
- Security vulnerabilities
- Performance degradation (>50% slower)
- User complaints (>10% negative feedback)

Success Metrics

User Experience

- Task completion rate: Maintain or improve
- Time to complete tasks: Maintain or improve
- User satisfaction: >4.0/5.0

Technical

- Page load time: <3s
- Time to interactive: <3s
- Accessibility score: 100% (axe)
- Performance score: >90 (Lighthouse)

Adoption

- Feature usage: Maintain or improve
- Error rate: Decrease
- Support tickets: Decrease

Resources

Documentation

- TPI UI/UX System
- Component Inventory
- Page Templates
- Pattern Guides

Tools

- axe DevTools - Accessibility testing
- Lighthouse - Performance testing
- Storybook - Component development

Support

Questions or Issues?

- Review TPI documentation
- Check component examples
- Contact TPI team lead

Next Steps:

1. Review this guide with team
2. Create implementation tickets
3. Begin Phase 1: Foundation
4. Track progress in project management tool